

```

// At single digits, look up the products in the multiplication table:
if ((x.length === 1) && (y.length === 1)) { // BASE CASE
    document.write("Lookup " + x.toString() + " * " + y.toString() + " = " +
        MULT_TABLE[[parseInt(x), parseInt(y)]] + "<br />");
    return MULT_TABLE[[parseInt(x), parseInt(y)]];
}

// RECURSIVE CASE
document.write("Multiplying " + x.toString() + " * " + y.toString() +
    "<br />");

// Pad with prepended zeros so that x and y are the same length:
if (x.length < y.length) { ❷
    // If x is shorter than y, pad x with zeros:
    x = padZeros(x, y.length - x.length, "left");
} else if (y.length < x.length) {
    // If y is shorter than x, pad y with zeros:
    y = padZeros(y, x.length - y.length, "left");
}
// At this point, x and y have the same length.

let halfOfDigits = Math.floor(x.length / 2); ❸

// Split x into halves a & b, split y into halves c & d:
let a = parseInt(x.substring(0, halfOfDigits));
let b = parseInt(x.substring(halfOfDigits));
let c = parseInt(y.substring(0, halfOfDigits));
let d = parseInt(y.substring(halfOfDigits));

// Make the recursive calls with these halves:
let step1Result = karatsuba(a, c); ❹ // Step 1: Multiply a & c.
let step2Result = karatsuba(b, d); // Step 2: Multiply b & d.
let step3Result = karatsuba(a + b, c + d); // Step 3: Multiply a + b & c + d.

// Step 4: Calculate Step 3 - Step 2 - Step 1:
let step4Result = step3Result - step2Result - step1Result; ❺

// Step 5: Pad these numbers, then add them for the return value:
let step1Padding = (x.length - halfOfDigits) + (x.length - halfOfDigits);
let step1PaddedNum = parseInt(padZeros(step1Result.toString(), step1Padding,
    "right"));

let step4Padding = (x.length - halfOfDigits);
let step4PaddedNum = parseInt(padZeros((step4Result).toString(), step4Paddin

```